

# **FUNCIONAMIENTO DE WEB SERVICES REST**

## **Proyecto Mapuescuela - Grupo 4**

Documento de pruebas con Insomnia y persistencia en MySQL

---

Java + Jakarta REST/Jersey + JPA + GlassFish + MySQL + Insomnia

# 1. Objetivo del documento

Este documento reúne las pruebas más recientes realizadas sobre los Web Services REST del proyecto Mapuescuola

Las pruebas se hicieron desde Insomnia, mientras que MySQL Workbench se utilizó para revisar directamente los datos almacenados. De esta forma se puede comparar lo que responde la API con lo que realmente queda registrado en la base de datos.

# 2. Arquitectura utilizada

La aplicación mantiene una estructura sencilla. Insomnia funciona como cliente de prueba, GlassFish ejecuta la aplicación Java, Jersey recibe las peticiones REST y JPA se encarga de conectar la aplicación con MySQL.

| Componente            | Función                                                          |
|-----------------------|------------------------------------------------------------------|
| Insomnia              | Envía las peticiones HTTP y permite revisar las respuestas JSON. |
| GlassFish             | Ejecuta la aplicación Mapuescuola en el puerto 8080.             |
| Jakarta REST / Jersey | Recibe y dirige las peticiones GET, POST y PUT.                  |
| JPA / EntityManager   | Gestiona las operaciones de persistencia desde Java.             |
| MySQL                 | Almacena los pedidos de manera permanente en la tabla pedido.    |

El flujo general de una petición queda de la siguiente forma:

```
Insomnia
  ↓ HTTP
GlassFish :8080
  ↓
/mapuescuola-api/webapi
  ↓
PedidoResource.java
  ↓ JPA / EntityManager
MySQL - tabla pedido
  ↓
Respuesta JSON
```

# 3. Ruta base y endpoints probados

La ruta principal usada durante las pruebas fue:

```
http://localhost:8080/mapuescuola-api/webapi/pedidos
```

| Método | Endpoint             | Uso                            | Resultado esperado       |
|--------|----------------------|--------------------------------|--------------------------|
| POST   | /pedidos             | Crear un pedido                | 201 Created              |
| GET    | /pedidos             | Listar todos los pedidos       | 200 OK                   |
| GET    | /pedidos/{id}        | Buscar un pedido por su ID     | 200 OK / 404 Not Found   |
| PUT    | /pedidos/{id}/estado | Cambiar el estado de un pedido | 200 OK / 400 Bad Request |

## 4. Prueba de creación de pedido con POST

Primero se creó un pedido nuevo desde Insomnia. En el cuerpo de la petición se enviaron solamente los datos que debe ingresar el cliente: nombre, producto y modalidad de entrega. El ID y el estado inicial son controlados por el backend.

```
{
  "cliente": "Dylan Perez",
  "producto": "Producto Mapuescuola",
  "modalidadEntrega": "RETIRO"
}
```

La respuesta fue 201 Created. El servidor creó el pedido con ID 1 y asignó automáticamente el estado PENDIENTE\_PAGO. Con esto se confirma que el endpoint POST está funcionando y que el pedido se genera con los datos esperados.

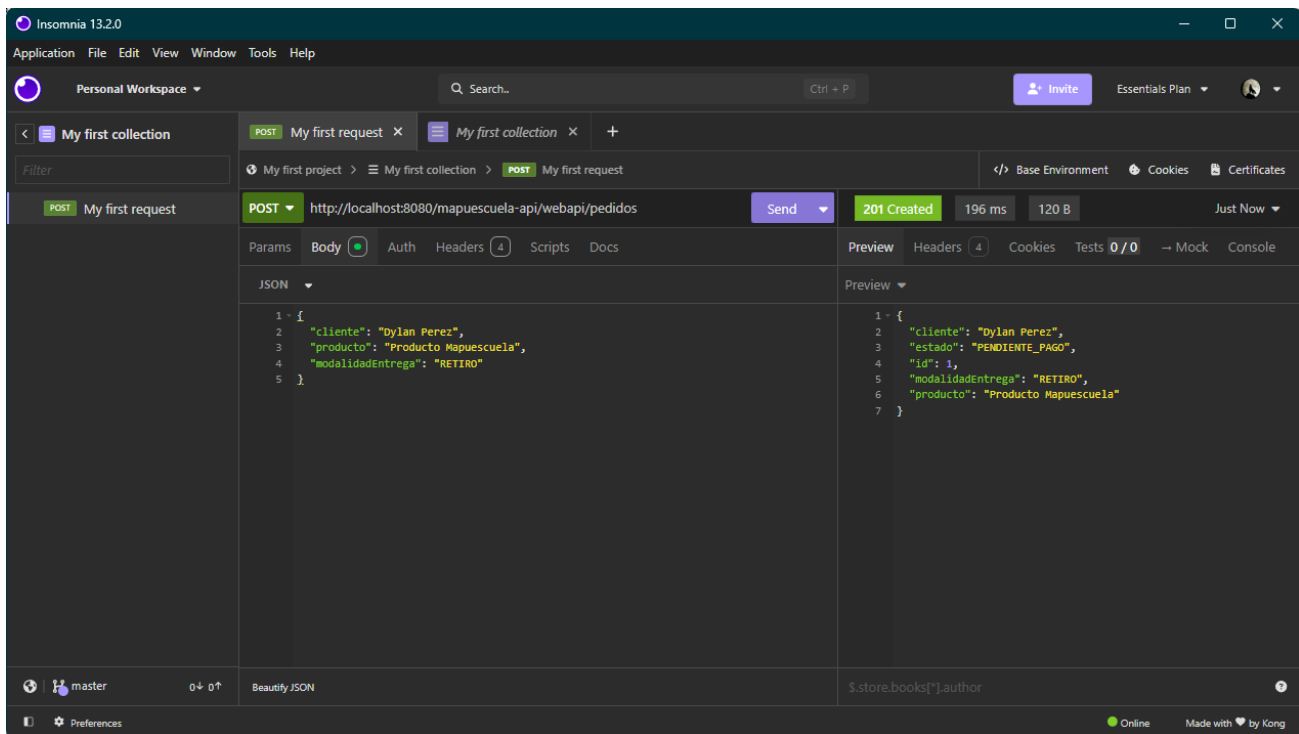


Figura 1. Creación del pedido mediante POST. Insomnia responde 201 Created y devuelve el pedido con estado PENDIENTE\_PAGO.

## 5. Comprobación del POST en MySQL

Después de crear el pedido se revisó la tabla pedido desde MySQL Workbench. Allí aparece el mismo registro con ID 1, cliente Dylan Perez, estado PENDIENTE\_PAGO, modalidad RETIRO y el producto indicado en la solicitud.

Esta comprobación es importante porque muestra que el pedido no quedó solamente en la respuesta de Insomnia, si no que también fué guardado en la base de datos.

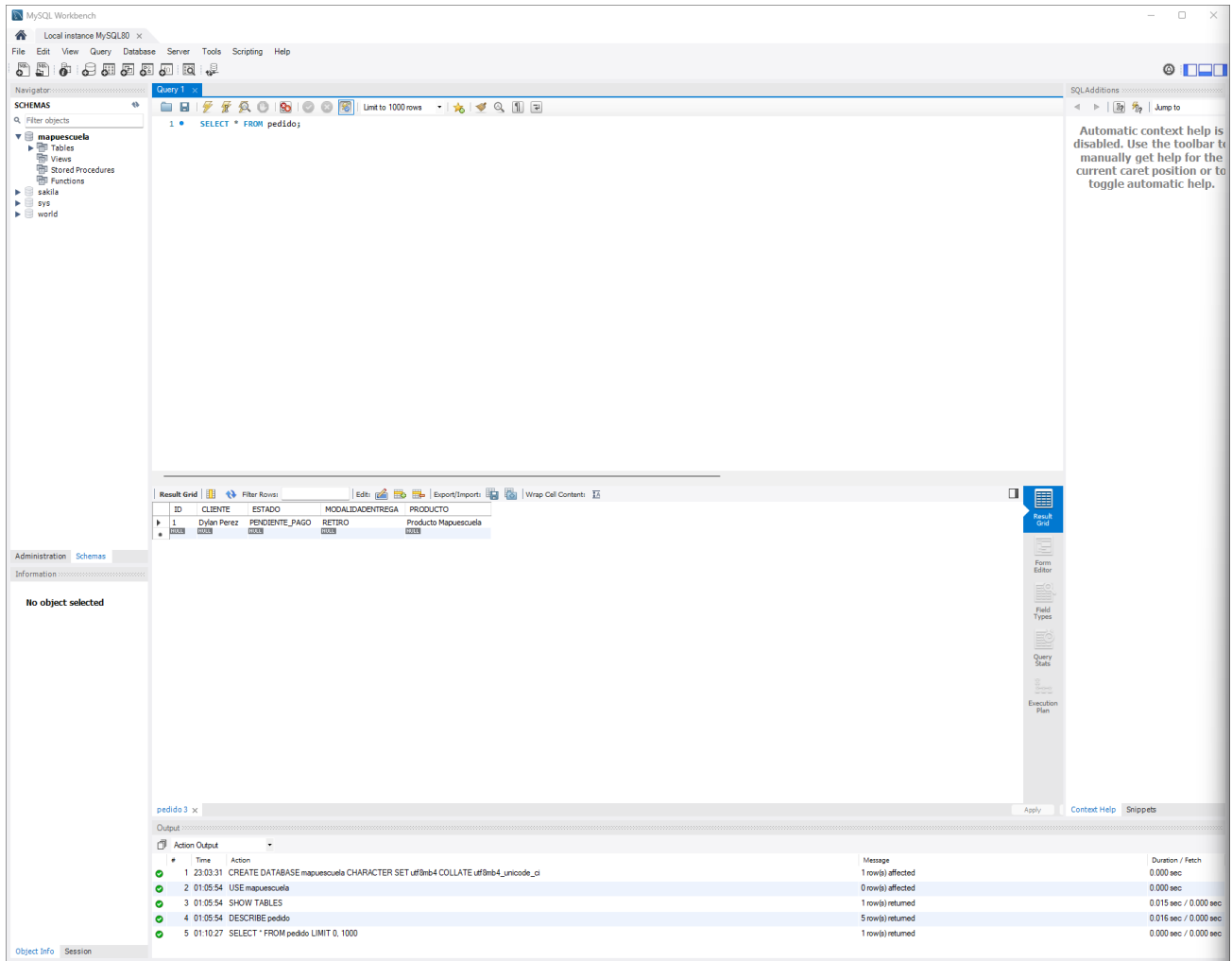


Figura 2. Registro del pedido en MySQL después del POST. El estado almacenado inicialmente es PENDIENTE\_PAGO.

## 6. Consulta general de pedidos con GET

Con el pedido ya creado, se ejecutó un GET sobre /pedidos. La API respondió 200 OK y devolvió la lista de pedidos registrados. En este caso aparece el pedido con ID 1 y los mismos datos almacenados previamente.

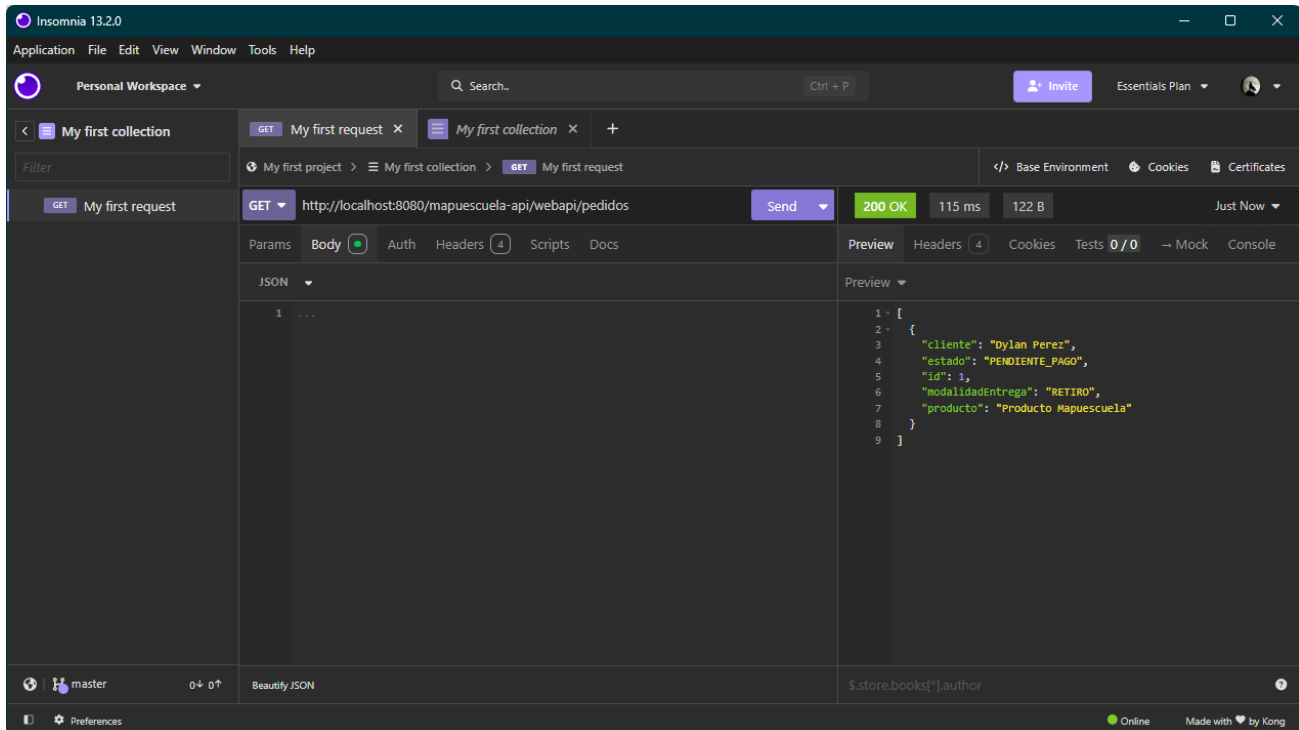


Figura 3. GET /pedidos devuelve la lista de pedidos existentes con respuesta 200 OK.

La respuesta coincide con lo observado en MySQL, por lo que la consulta general está leyendo correctamente la información persistida.

## 7. Consulta de un pedido por ID

Luego se probó la búsqueda de un pedido específico utilizando su identificador. Al consultar /pedidos/1, la API respondió 200 OK y entregó únicamente el pedido correspondiente al ID 1.

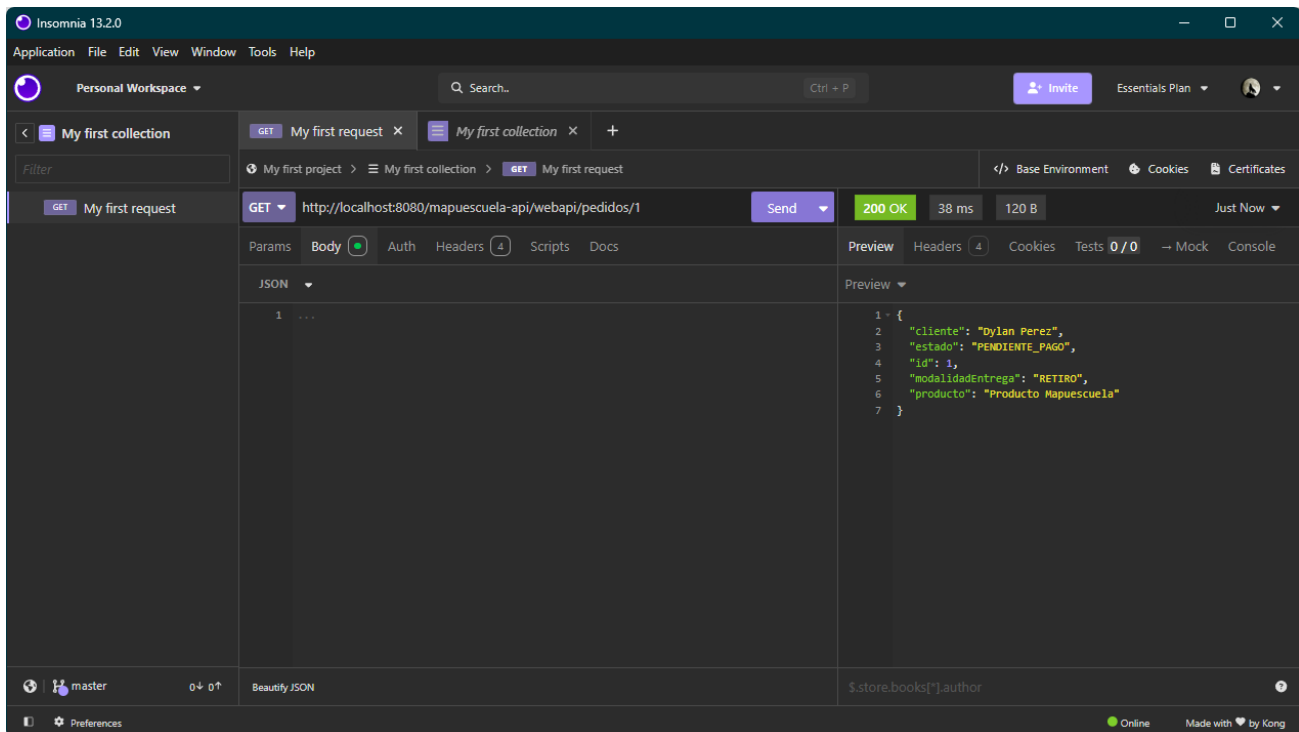


Figura 4. GET /pedidos/1 devuelve correctamente el pedido con ID 1.

## 8. Manejo de un ID que no existe

También se probó qué ocurre cuando se solicita un pedido inexistente. Para eso se consultó el ID 9999. En lugar de devolver un objeto vacío o generar un error del servidor, la API respondió 404 Not Found con el mensaje “Pedido no encontrado” lo que permite informar de forma clara que el recurso solicitado no existe y evita confundir este caso con una respuesta válida.

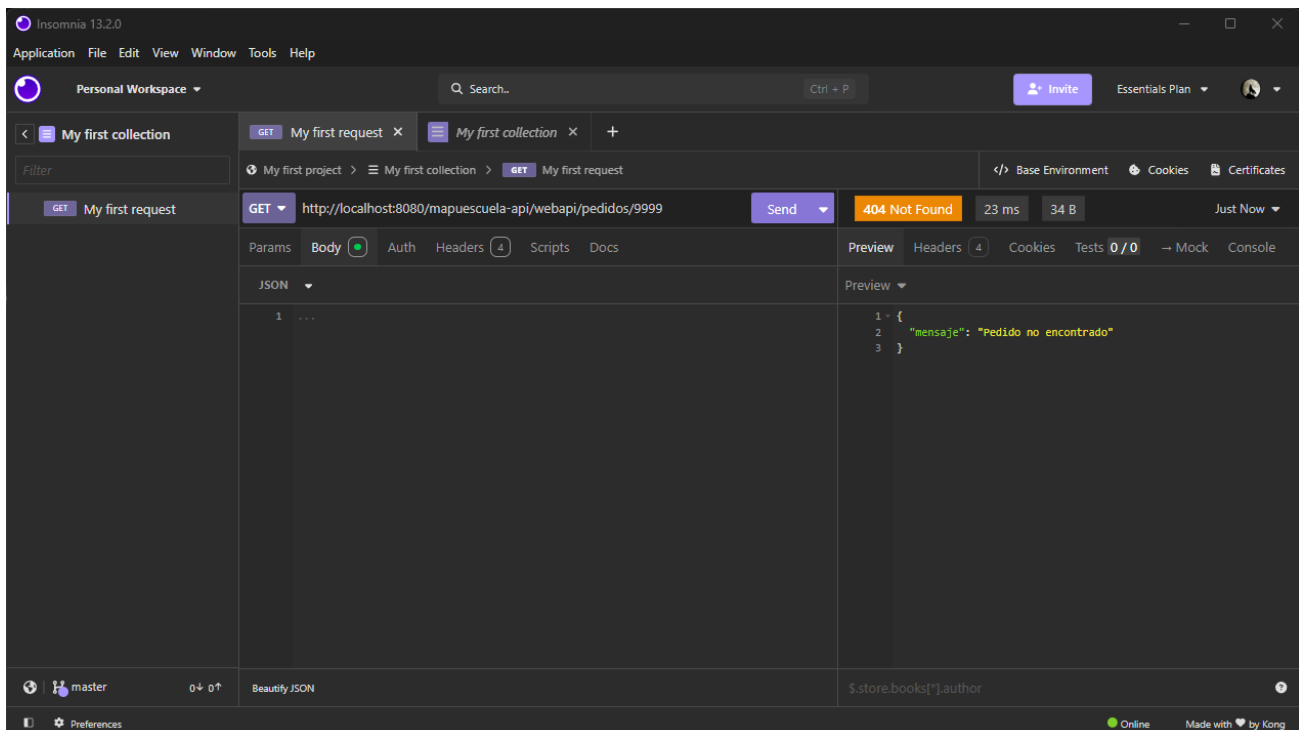


Figura 5. Prueba de error con GET /pedidos/9999. La API responde 404 Not Found.

## 9. Actualización del estado con PUT

Para probar la actualización del pedido se utilizó el endpoint PUT /pedidos/1/estado. Se envió el nuevo estado PAGADO en formato JSON.

```
{
  "estado": "PAGADO"
}
```

La API respondió 200 OK y devolvió el pedido actualizado. El resto de los datos se mantuvo sin cambios y solamente se modificó el estado, que pasó de PENDIENTE\_PAGO a PAGADO.

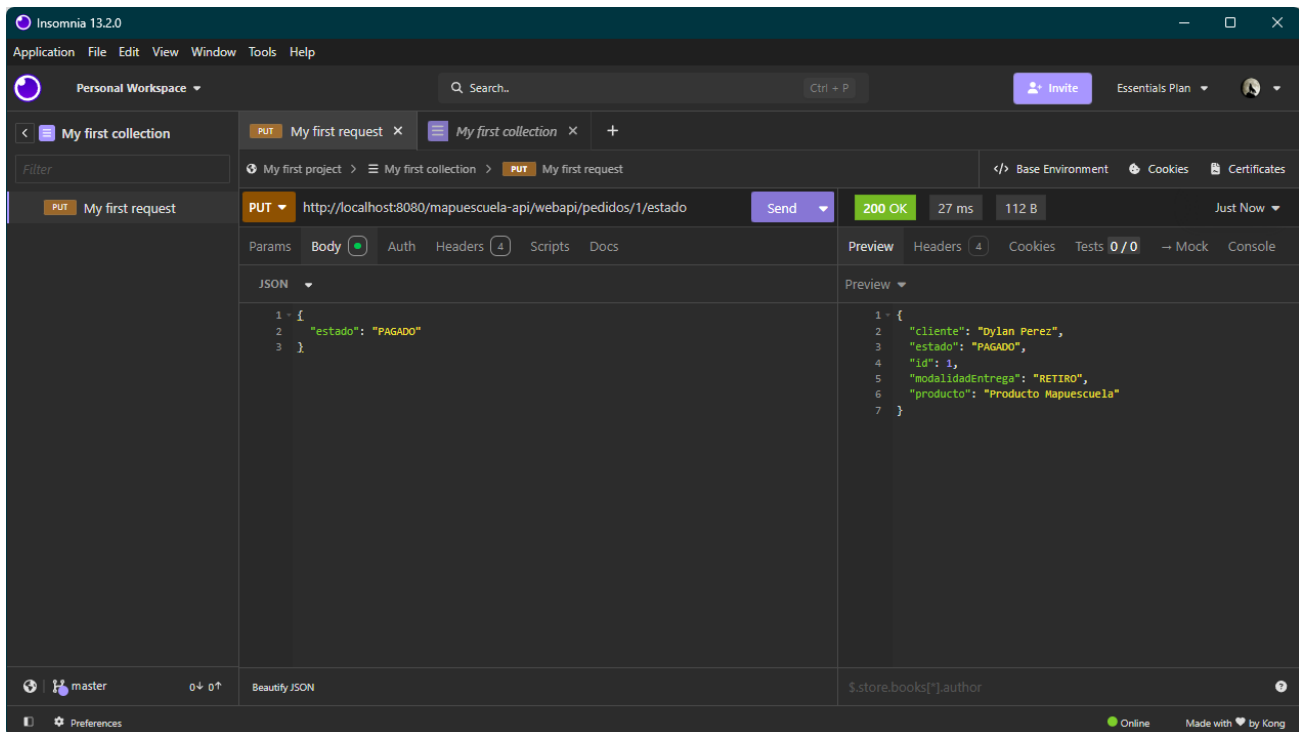


Figura 6. Actualización del pedido mediante PUT. El estado cambia a PAGADO y la API responde 200 OK.

## 10. Persistencia del cambio en MySQL

Después del PUT se ejecutó nuevamente la consulta en MySQL Workbench. El mismo pedido con ID 1 aparece después con el estado PAGADO, lo que confirma que la modificación fue escrita en la base de datos.

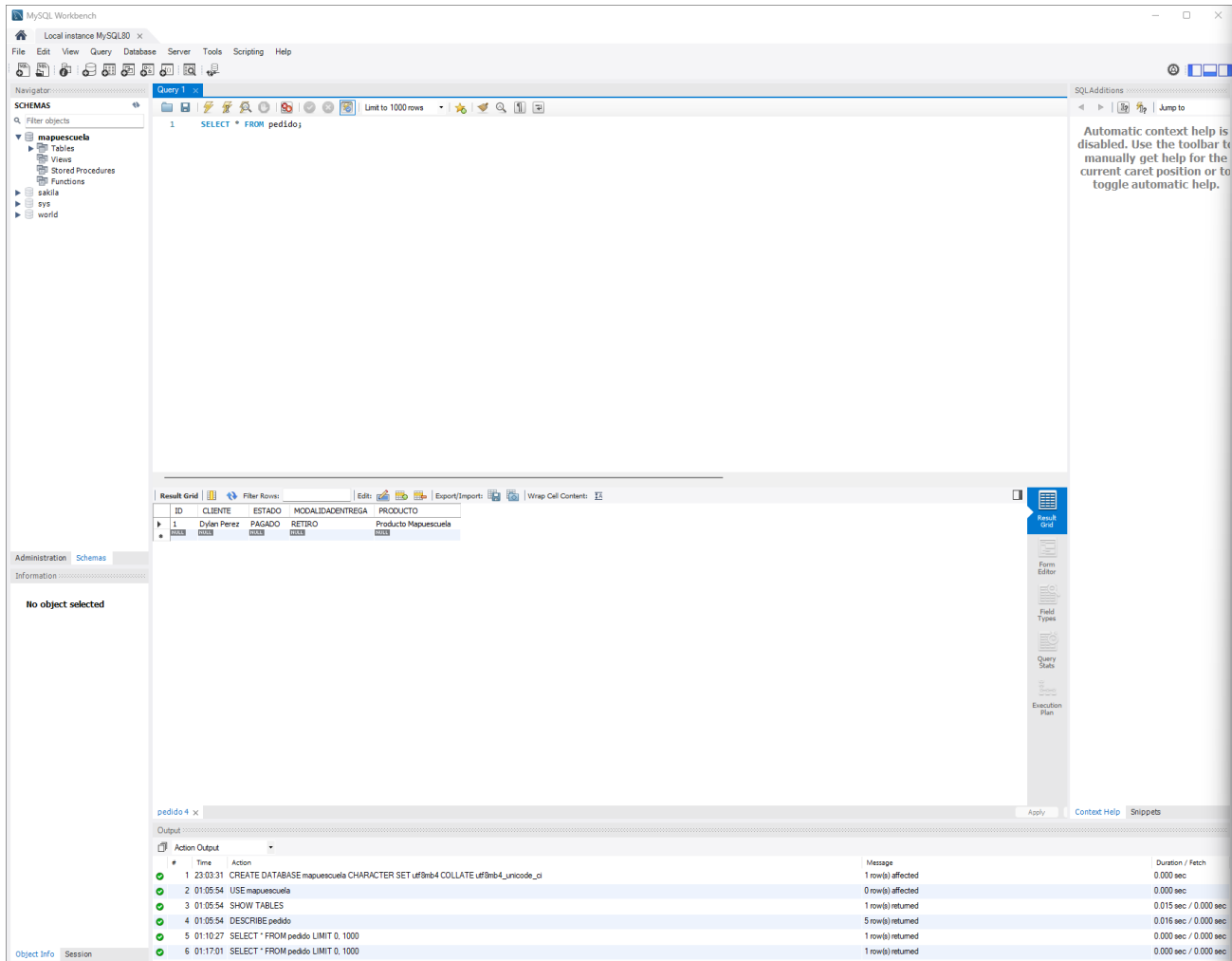


Figura 7. MySQL muestra el pedido con estado PAGADO después de la actualización realizada desde Insomnia.

Con esta prueba se valida que las modificaciones realizadas a través de la API se conservan en MySQL y no se pierden al terminar una petición.

## 11. Validación de datos obligatorios en PUT

Por último, se probó el mismo endpoint de actualización enviando el campo estado vacío. Esta vez la API respondió 400 Bad Request y mostró el mensaje “Debe indicar el estado”.

```
{  
  "estado": ""  
}
```

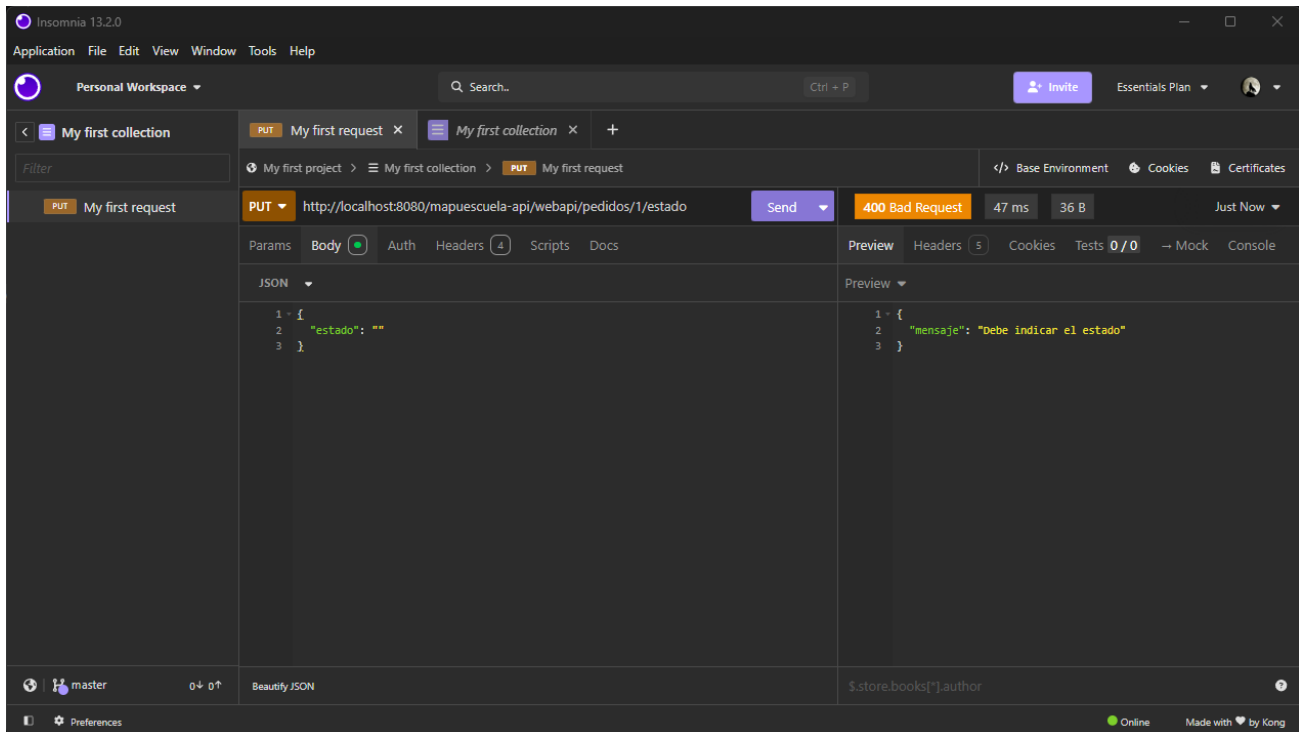


Figura 8. PUT con el estado vacío. La API rechaza la solicitud con 400 Bad Request.

Esta validación evita guardar un estado vacío y permite que el cliente reciba un mensaje entendible sobre el dato que falta. Esta prueba demuestra que el servicio no acepta cualquier contenido sin revisarlo antes.

## 12. Códigos HTTP observados

| Código          | Qué ocurrió en la prueba                            | Ejemplo                         |
|-----------------|-----------------------------------------------------|---------------------------------|
| 200 OK          | La operación se realizó correctamente.              | GET de pedidos y PUT de estado. |
| 201 Created     | Se creó un pedido nuevo.                            | POST /pedidos.                  |
| 400 Bad Request | La solicitud llegó, pero faltó un dato obligatorio. | PUT con estado vacío.           |
| 404 Not Found   | El pedido solicitado no existe.                     | GET /pedidos/9999.              |

## 13. Resultado general de las pruebas

Las pruebas realizadas muestran que el servicio de pedidos está respondiendo de acuerdo con lo esperado. Se logró crear un pedido, consultarlo de forma general y por ID, actualizar su estado y comprobar el cambio directamente en MySQL. También se probaron dos situaciones de error: buscar un pedido inexistente y enviar un estado vacío.

La diferencia principal respecto de la versión inicial es que ahora existe persistencia real en MySQL.